

Volume 3 – Agent SDK Developer Guide

Version: 1.0

Purpose

This document defines the Agent SDK used by developers to create, test, deploy, and maintain agents within the Enterprise Tiny Agent Framework.

This guide focuses on:

- Creating new agents
- Implementing agent lifecycle hooks
- Planning and execution
- Tool integration
- Model integration
- Agent testing
- Deployment standards

Audience:

- Java Developers
 - Platform Engineers
 - AI Engineers
 - Solution Architects
-

Agent Development Philosophy

An Agent should contain:

```
Business Capability
+
Decision Logic
+
Tool Orchestration
```

An Agent should NOT contain:

```
Database Logic
Filesystem Logic
REST Integration Logic
Business Data Access Logic
```

Those belong inside Tools.

Agent Development Lifecycle

Every Agent follows:

```
Create Agent Definition
      ↓
Implement Agent
      ↓
Register Agent
      ↓
Configure Models
      ↓
Configure Tools
      ↓
Test
      ↓
Deploy
```

Agent Package Structure

Recommended:

```
com.company.agent
├── api
│
├── definition
│
├── implementation
│
├── planner
│
└──
```

```
|— response
|
|— validation
|
|— test
```

Example:

```
sql-agent

|— definition
|   SqlAgentDefinition
|
|— implementation
|   SqlAgent
|
|— planner
|   SqlPlanner
|
|— validation
|   SqlRequestValidator
|
|— test
```

Agent Definition

Every Agent must have metadata.

```
public class AgentDefinition {

    private String id;

    private String name;

    private String description;

    private List<String> capabilities;

    private List<String> allowedTools;

    private String preferredModel;
```

```
private String fallbackModel;

private Double confidenceThreshold;

}
```

Example:

```
@Bean
public AgentDefinition sqlAgentDefinition() {

    return AgentDefinition.builder()
        .id("sql-agent")
        .name("SQL Agent")
        .description("Generates SQL")
        .preferredModel("qwen-4b")
        .fallbackModel("qwen-14b")
        .confidenceThreshold(0.80)
        .build();

}
```

Agent Registration

Agents should be auto-discovered.

```
@Component
public class SqlAgent extends AbstractAgent {

}
```

Framework startup:

```
List<Agent> agents =
    applicationContext.getBeansOfType(
        Agent.class);
```

Register automatically:

```
AgentRegistry.register(agent);
```

Abstract Agent

All agents extend:

```
public abstract class AbstractAgent
    implements Agent {
}
```

Developers implement only business-specific logic.

Lifecycle Hooks

initialize()

Purpose:

Prepare execution.

```
protected AgentExecutionContext initialize(
    AgentRequest request)
```

Typical actions:

```
Generate Request ID

Load Agent Definition

Load Session
```

validate()

Purpose:

Reject invalid requests.

Example:

```
protected void validate(
    AgentExecutionContext context) {

    if(context.getUserInput() == null) {
        throw new ValidationException();
    }
}
```

buildContext()

Purpose:

Prepare working context.

Example:

```
protected void buildContext(
    AgentExecutionContext context) {

    context.put(
        "requestTime",
        Instant.now()
    );
}
```

Avoid heavy retrieval here.

Use retrieval only when necessary.

plan()

Purpose:

Create execution plan.

Output:

```
AgentPlan
```

Example:

```
protected AgentPlan plan(  
    AgentExecutionContext context) {  
  
    return AgentPlan.builder()  
        .addAction("searchMetadata")  
        .addAction("generateSql")  
        .build();  
}
```

executePlan()

Purpose:

Execute actions.

Example:

```
protected AgentExecutionResult executePlan(  
    AgentExecutionContext context,  
    AgentPlan plan) {  
  
    Tool tool =  
        toolRegistry.getTool(  
            "searchMetadata");  
  
    ToolResult result =  
        tool.execute(...);  
  
    return AgentExecutionResult.of(result);  
}
```

generateResponse()

Purpose:

Convert execution result into final output.

Example:

```
protected AgentResponse generateResponse(  
    AgentExecutionContext context,  
    AgentExecutionResult result) {  
  
    return AgentResponse.success(  
        result.getData());  
}
```

audit()

Purpose:

Record execution details.

Example:

```
protected void audit(  
    AgentExecutionContext context,  
    AgentExecutionResult result) {  
  
    auditService.record(  
        context,  
        result);  
}
```

Agent Planning

Planning should be simple.

Do:

```
Request  
  ↓  
Actions
```

Avoid:

Recursive Planning

Self Reflection

Autonomous Reasoning Loops

For enterprise systems, deterministic planning wins.

Agent Plan Structure

```
public class AgentPlan {  
    private List<PlannedAction> actions;  
}
```

Planned Action

```
public class PlannedAction {  
    private String actionId;  
    private String toolName;  
    private Map<String, Object> parameters;  
}
```

Example:

```
new PlannedAction(  
    "search",  
    "searchDataset",  
    params  
);
```

Agent Context

Agent Context is the execution state.

```
public class AgentExecutionContext {  
  
    private String requestId;  
  
    private String agentId;  
  
    private String sessionId;  
  
    private String userInput;  
  
    private Map<String,Object> attributes;  
  
    private AgentMemory memory;  
  
}
```

Rules:

- No static state
- No ThreadLocal state
- No hidden dependencies

Agent Output

Every Agent should return structured output.

Bad:

```
I found three tables...
```

Good:

```
{  
  "tables": [  
    {  
      "name": "subscriber_usage"  
    }  
  ]  
}
```

```
]
}
```

Natural language can be generated later.

Tool Access

Agents must only access approved tools.

Framework validation:

```
if (!agentDefinition
    .getAllowedTools()
    .contains(toolName)) {

    throw new SecurityException();
}
```

This prevents accidental privilege escalation.

Model Usage

Agents never instantiate models directly.

Bad:

```
new OllamaClient(...)
```

Good:

```
ModelProvider provider =
    modelRouter.route(context);
```

Benefits:

- Vendor independence
 - Easier testing
 - Dynamic routing
-

Confidence Scoring

Every Agent should estimate confidence.

Example:

```
AgentResponse.builder()  
    .confidence(0.92)  
    .build();
```

Usage:

```
confidence > 0.8  
    return  
  
confidence < 0.8  
    escalate
```

Escalation

Supported escalation strategies:

```
Retry  
  
Use Larger Model  
  
Request Clarification  
  
Human Review
```

Example:

```
if(response.getConfidence() < 0.80) {  
    return escalationService.escalate(  
        context);  
}
```

Agent Configuration

Externalized YAML.

```
agent:
  id: sql-agent
  preferred-model: qwen-4b
  fallback-model: qwen-14b
  confidence-threshold: 0.80
  allowed-tools:
    - metadata-search
    - calcite-validator
```

Avoid hardcoding.

Agent Testing

Unit Tests

Test:

```
Validation
Planning
Response Generation
```

Mock:

```
Tools
Models
Memory
```

Integration Tests

Test:

```
Agent
+
Real Tools
```

Mock only external systems.

Golden Dataset Tests

Maintain:

```
{
  "input": "Find subscriber usage dataset",
  "expectedTool": "searchDataset",
  "expectedAgent": "metadata-agent"
}
```

Run continuously.

Agent Metrics

Every Agent should publish:

```
Requests

Failures

Execution Time

Tool Calls

Tokens Used

Cost

Confidence
```

Useful for:

- Optimization
- Capacity Planning
- Model Evaluation

Agent Versioning

Version independently.

Example:

```
sql-agent-v1
```

```
sql-agent-v2
```

Never modify behavior silently.

Agent Development Checklist

Before production:

- Single responsibility verified
 - Allowed tools defined
 - Confidence threshold defined
 - Unit tests complete
 - Integration tests complete
 - Audit events enabled
 - Metrics enabled
 - Configuration externalized
 - Security review completed
-

Example SQL Agent

Capability:

```
Generate SQL from metadata
```

Allowed Tools:

Metadata Search
Schema Lookup
Calcite Validation

Preferred Model:

Qwen 4B

Fallback:

Qwen 14B

Output:

```
{  
  "sql": "SELECT * FROM subscriber_usage",  
  "validated": true,  
  "confidence": 0.91  
}
```

No execution.

Execution belongs to another tool or agent.

Final Rule

An Agent should be easy to understand within 15 minutes.

If a developer needs hours to understand:

- its purpose,
- its tools,
- its execution flow,

then the agent is too large and should be split into smaller capability-focused agents.